

Accenture Most Asked Java DSA Questions

(Covers Arrays, Strings, Linked List, Stack, Queue, Trees, BST, DP & More)

Created by: Abhishek Rathor

Instagram: `SYNTAX ERROR` (code.abhii07)

Key Features:

- **75+ Handpicked Interview Questions** from real coding rounds & assessments.
- **Beginner to Advanced Topics:**
 - Arrays → Strings → Linked List → Stack → Queue → Trees → BST → Dynamic Programming → Advanced Backtracking & Trie.
- **Java Implementations** with well-structured **Time & Space Complexities**.
- **Concept Explanations** and **Optimal Approaches** for quick revision.
- **Optimized Code** and patterns used in top product & service-based companies.

Why This Guide?

- ☐ **Save Time:** No need to browse scattered sources — this is your all-in-one prep resource.
- **Practice Smart:** Focused on **most frequently asked DSA problems**
- **Understand Concepts:** Each problem includes logic, patterns, dry-run approach, and edge cases.
- **Placement Ready:** Aligned with Coding Rounds (Online + Technical Interview Levels).

☐ *“Success isn’t just about writing code — it’s about solving problems effectively.”*

Arrays

LeetCode Problem #1

1. Two Sum

Problem Statement:

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Solution Logic:

Use a hashmap to store each number and its index. For every element, check if `target - nums[i]` exists in the hashmap.

Java Code:

```
import java.util.HashMap;

public int[] twoSum(int[] nums, int target) {
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[0]; // no solution
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

LeetCode Problem #26

2. Remove Duplicates from Sorted Array

Problem Statement:

Given a sorted array `nums`, remove the duplicates in-place such that each element appears only once and return the new length.

Solution Logic:

Use two pointers: one to traverse and one to place unique elements.

Java Code:

```

public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int i = 0;
    for (int j = 1; j < nums.length; j++) {
        if (nums[j] != nums[i]) {
            i++;
            nums[i] = nums[j];
        }
    }
    return i + 1;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

LeetCode Problem #33

3. Search in Rotated Sorted Array

Problem Statement:

Given a rotated sorted array and a target, find the index of target in the array or return -1.

Solution Logic:

Apply modified binary search considering pivot rotation.

Java Code:

```

public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;

        if (nums[left] <= nums[mid]) { // left part sorted
            if (target >= nums[left] && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        } else { // right part sorted
            if (target > nums[mid] && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}

```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

LeetCode Problem #27

4. Remove Element

Problem Statement:

Remove all instances of a value in-place and return the new length of the array.

Solution Logic:

Use two pointers, one for traversing and one for placement of non-val elements.

Java Code:

```
public int removeElement(int[] nums, int val) {
    int i = 0;
    for (int j = 0; j < nums.length; j++) {
        if (nums[j] != val) {
            nums[i++] = nums[j];
        }
    }
    return i;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

LeetCode Problem #53

5. Maximum Subarray

Problem Statement:

Find the contiguous subarray with the largest sum.

Solution Logic:

Use Kadane's Algorithm: keep track of current sum and max sum.

Java Code:

```
public int maxSubArray(int[] nums) {
    int maxSoFar = nums[0], maxEndingHere = nums[0];
    for (int i = 1; i < nums.length; i++) {
        maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);
        maxSoFar = Math.max(maxSoFar, maxEndingHere);
    }
    return maxSoFar;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Sure! Continuing now with **Strings** (questions 6 to 15), with concept heading and clean formatting, all in Java:

Strings

LeetCode Problem #242

6. Valid Anagram

Problem Statement:

Given two strings s and t , write a function to determine if t is an anagram of s .

Solution Logic:

Count frequency of each character using arrays or hashmaps and compare counts.

Java Code:

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] counts = new int[26];
    for (int i = 0; i < s.length(); i++) {
        counts[s.charAt(i) - 'a']++;
        counts[t.charAt(i) - 'a']--;
    }
    for (int count : counts) {
        if (count != 0) return false;
    }
    return true;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

LeetCode Problem #49

7. Group Anagrams

Problem Statement:

Given an array of strings, group anagrams together.

Solution Logic:

Use a hashmap where the key is the sorted string, and values are lists of anagrams.

Java Code:

```
import java.util.*;

public List<List<String>> groupAnagrams(String[] strs) {
    if (strs == null || strs.length == 0) return new ArrayList<>();
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] arr = s.toCharArray();
        Arrays.sort(arr);
        String key = new String(arr);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

Time Complexity: $O(n * k \log k)$ where k is max string length

Space Complexity: $O(nk)$

LeetCode Problem #125

8. Valid Palindrome

Problem Statement:

Given a string, determine if it is a palindrome considering only alphanumeric characters and ignoring cases.

Solution Logic:

Use two pointers from start and end, skip non-alphanumeric, and compare characters.

Java Code:

```
public boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
        if (Character.toLowerCase(s.charAt(left)) !=
            Character.toLowerCase(s.charAt(right))) return false;
        left++; right--;
    }
    return true;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

LeetCode Problem #5

9. Longest Palindromic Substring

Problem Statement:

Given a string *s*, return the longest palindromic substring.

Solution Logic:

Expand around center for each index to find palindromes.

Java Code:

```
public String longestPalindrome(String s) {
    if (s == null || s.length() < 1) return "";
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expandAroundCenter(s, i, i);
        int len2 = expandAroundCenter(s, i, i + 1);
        int len = Math.max(len1, len2);
        if (len > end - start) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }
    return s.substring(start, end + 1);
}

private int expandAroundCenter(String s, int left, int right) {
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        left--;
        right++;
    }
    return right - left - 1;
}
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

LeetCode Problem #14

10. Longest Common Prefix

Problem Statement:

Write a function to find the longest common prefix string amongst an array of strings.

Solution Logic:

Compare characters of all strings one by one until mismatch.

Java Code:

```

public String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";
    for (int i = 0; i < strs[0].length(); i++) {
        char c = strs[0].charAt(i);
        for (int j = 1; j < strs.length; j++) {
            if (i == strs[j].length() || strs[j].charAt(i) != c) {
                return strs[0].substring(0, i);
            }
        }
    }
    return strs[0];
}

```

Time Complexity: $O(n * m)$ where n = number of strings, m = min string length

Space Complexity: $O(1)$

LeetCode Problem #38

11. Count and Say

Problem Statement:

The count-and-say sequence is a sequence of digit strings defined by describing the previous string.

Solution Logic:

Iteratively build each string by counting consecutive characters.

Java Code:

```

public String countAndSay(int n) {
    if (n == 1) return "1";
    String prev = countAndSay(n - 1);
    StringBuilder sb = new StringBuilder();
    int count = 1;
    for (int i = 1; i < prev.length(); i++) {
        if (prev.charAt(i) == prev.charAt(i - 1)) {
            count++;
        } else {
            sb.append(count).append(prev.charAt(i - 1));
            count = 1;
        }
    }
    sb.append(count).append(prev.charAt(prev.length() - 1));
    return sb.toString();
}

```

Time Complexity: $O(2^n)$ approximately (sequence length doubles)

Space Complexity: $O(2^n)$

LeetCode Problem #28

12. Implement strStr()

Problem Statement:

Return the index of the first occurrence of `needle` in `haystack`, or -1 if not found.

Solution Logic:

Use simple sliding window or KMP algorithm (simple version here).

Java Code:

```
public int strStr(String haystack, String needle) {
    if (needle.isEmpty()) return 0;
    int hLen = haystack.length(), nLen = needle.length();
    for (int i = 0; i <= hLen - nLen; i++) {
        if (haystack.substring(i, i + nLen).equals(needle)) {
            return i;
        }
    }
    return -1;
}
```

Time Complexity: $O((h-n+1)*n)$

Space Complexity: $O(1)$

LeetCode Problem #8

13. String to Integer (atoi)

Problem Statement:

Implement `atoi` to convert a string to a 32-bit signed integer.

Solution Logic:

Parse string, handle whitespaces, signs, overflow, and invalid characters.

Java Code:

```
public int myAtoi(String s) {
    int i = 0, sign = 1, result = 0;
    if (s.length() == 0) return 0;

    while (i < s.length() && s.charAt(i) == ' ') i++;

    if (i < s.length() && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
        sign = s.charAt(i) == '-' ? -1 : 1;
        i++;
    }

    while (i < s.length() && Character.isDigit(s.charAt(i))) {
        int digit = s.charAt(i) - '0';
        if (result > (Integer.MAX_VALUE - digit) / 10) {
            return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
        }
        result = result * 10 + digit * sign;
        i++;
    }

    return result;
}
```

```

        return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
    }
    result = result * 10 + digit;
    i++;
}
return result * sign;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

LeetCode Problem #20

14. Valid Parentheses

Problem Statement:

Given a string containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

Solution Logic:

Use stack to check for matching pairs.

Java Code:

```

import java.util.Stack;

public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push(')');
        else if (c == '{') stack.push('}');
        else if (c == '[') stack.push(']');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

LeetCode Problem #3

15. Longest Substring Without Repeating Characters

Problem Statement:

Find the length of the longest substring without repeating characters.

Solution Logic:

Sliding window with hashmap to track indices.

Java Code:

```
import java.util.HashMap;

public int lengthOfLongestSubstring(String s) {
    HashMap<Character, Integer> map = new HashMap<>();
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (map.containsKey(c)) {
            left = Math.max(map.get(c) + 1, left);
        }
        map.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(\min(m,n))$ where m = charset size

Linked Lists

16. LeetCode #21

Merge Two Sorted Lists**Problem:**

Given two sorted linked lists, merge them into one sorted list and return it.

Solution Logic:

Use two pointers to traverse both lists. Compare nodes and link smaller one first, continue until both lists are fully merged.

Java Code:

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    if (l1 == null) return l2;
    if (l2 == null) return l1;

    if (l1.val < l2.val) {
        l1.next = mergeTwoLists(l1.next, l2);
        return l1;
    } else {
        l2.next = mergeTwoLists(l1, l2.next);
        return l2;
    }
}
```

Time Complexity: $O(n + m)$ where n and m are lengths of the two lists

Space Complexity: $O(n + m)$ due to recursion stack (can be optimized iteratively)

17. LeetCode #206

Reverse Linked List

Problem:

Reverse a singly linked list.

Solution Logic:

Iterate through the list and reverse the next pointers one by one using three pointers: previous, current, next.

Java Code:

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;
    while (curr != null) {
        ListNode nextTemp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

18. LeetCode #141

Linked List Cycle

Problem:

Determine if a linked list has a cycle.

Solution Logic:

Use Floyd's cycle-finding algorithm (slow and fast pointers). If they meet, cycle exists.

Java Code:

```
public boolean hasCycle(ListNode head) {
    if (head == null) return false;
    ListNode slow = head, fast = head.next;
    while (fast != null && fast.next != null) {
        if (slow == fast) return true;
        slow = slow.next;
        fast = fast.next.next;
    }
    return false;
}
```

```
    }  
    return false;  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

19. LeetCode #142

Linked List Cycle II

Problem:

Find the node where the cycle begins in a linked list.

Solution Logic:

Use Floyd's algorithm to find meeting point. Then move one pointer to head, advance both one step at a time; meeting point is cycle start.

Java Code:

```
public ListNode detectCycle(ListNode head) {  
    if (head == null) return null;  
    ListNode slow = head, fast = head;  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
        if (slow == fast) {  
            ListNode ptr = head;  
            while (ptr != slow) {  
                ptr = ptr.next;  
                slow = slow.next;  
            }  
            return ptr;  
        }  
    }  
    return null;  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

20. LeetCode #24

Swap Nodes in Pairs

Problem:

Swap every two adjacent nodes in the list and return its head.

Solution Logic:

Iterate in pairs, swap nodes by adjusting next pointers carefully. Use dummy node for easy head swaps.

Java Code:

```
public ListNode swapPairs(ListNode head) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode current = dummy;
    while (current.next != null && current.next.next != null) {
        ListNode first = current.next;
        ListNode second = current.next.next;

        first.next = second.next;
        second.next = first;
        current.next = second;

        current = first;
    }
    return dummy.next;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

21. LeetCode #83

Remove Duplicates from Sorted List

Problem:

Delete duplicates such that each element appears only once in a sorted linked list.

Solution Logic:

Traverse list; if current node value equals next node, skip next node by adjusting next pointers.

Java Code:

```
public ListNode deleteDuplicates(ListNode head) {
    ListNode current = head;
    while (current != null && current.next != null) {
        if (current.val == current.next.val) {
            current.next = current.next.next;
        } else {
            current = current.next;
        }
    }
    return head;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

22. LeetCode #82

Remove Duplicates from Sorted List II

Problem:

Remove all nodes that have duplicate numbers, leaving only distinct numbers from the original list.

Solution Logic:

Use dummy node and two pointers to skip all duplicates completely.

Java Code:

```
public ListNode deleteDuplicates(ListNode head) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prev = dummy;
    while (head != null) {
        if (head.next != null && head.val == head.next.val) {
            while (head.next != null && head.val == head.next.val) {
                head = head.next;
            }
            prev.next = head.next;
        } else {
            prev = prev.next;
        }
        head = head.next;
    }
    return dummy.next;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

23. LeetCode #160

Intersection of Two Linked Lists

Problem:

Find the node where two singly linked lists intersect or null if they do not intersect.

Solution Logic:

Use two pointers starting at heads of each list; switch heads when reaching list end. They will meet at intersection or null.

Java Code:

```
public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
    if (headA == null || headB == null) return null;
    ListNode a = headA, b = headB;
    while (a != b) {
```

```

        a = (a == null) ? headB : a.next;
        b = (b == null) ? headA : b.next;
    }
    return a;
}

```

Time Complexity: $O(m + n)$

Space Complexity: $O(1)$

24. LeetCode #61

Rotate List

Problem:

Rotate the list to the right by k places.

Solution Logic:

Make the list circular, find new tail after $n-k$ steps, break the circle.

Java Code:

```

public ListNode rotateRight(ListNode head, int k) {
    if (head == null || k == 0) return head;
    int length = 1;
    ListNode tail = head;
    while (tail.next != null) {
        tail = tail.next;
        length++;
    }
    k = k % length;
    if (k == 0) return head;

    tail.next = head;
    for (int i = 0; i < length - k; i++) {
        tail = tail.next;
    }
    ListNode newHead = tail.next;
    tail.next = null;
    return newHead;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

25. LeetCode #328

Odd Even Linked List

Problem:

Group all odd nodes together followed by even nodes.

Solution Logic:

Maintain two pointers for odd and even nodes, rearrange pointers accordingly.

Java Code:

```
public ListNode oddEvenList(ListNode head) {
    if (head == null) return null;
    ListNode odd = head, even = head.next, evenHead = even;
    while (even != null && even.next != null) {
        odd.next = even.next;
        odd = odd.next;
        even.next = odd.next;
        even = even.next;
    }
    odd.next = evenHead;
    return head;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Stacks & Queues

26. LeetCode #20

Valid Parentheses**Problem:**

Given a string containing only '(', ')', '{', '}', '[' and ']', determine if the input string is valid. A string is valid if open brackets are closed in the correct order.

Solution Logic:

Use a stack to store open brackets. For each closing bracket, check if the top of the stack matches its pair.

Java Code:

```
public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push('(');
        else if (c == '{') stack.push('{');
        else if (c == '[') stack.push('[');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

27. LeetCode #155

Min Stack

Problem:

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Solution Logic:

Use two stacks — one for values, one for tracking minimums. Push new min when current value $\leq$ min.

Java Code:

```
class MinStack {
    Stack<Integer> stack;
    Stack<Integer> minStack;

    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public void push(int val) {
        stack.push(val);
        if (minStack.isEmpty() || val <= minStack.peek())
            minStack.push(val);
    }

    public void pop() {
        if (stack.pop().equals(minStack.peek()))
            minStack.pop();
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}
```

Time Complexity: $O(1)$

Space Complexity: $O(n)$

28. LeetCode #232

Implement Queue using Stacks

Problem:

Implement a queue using two stacks.

Solution Logic:

Push elements into `inStack`. When popping, if `outStack` is empty, transfer all from `inStack` to `outStack`.

Java Code:

```
class MyQueue {
    Stack<Integer> inStack = new Stack<>();
    Stack<Integer> outStack = new Stack<>();

    public void push(int x) {
        inStack.push(x);
    }

    public int pop() {
        peek();
        return outStack.pop();
    }

    public int peek() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty())
                outStack.push(inStack.pop());
        }
        return outStack.peek();
    }

    public boolean empty() {
        return inStack.isEmpty() && outStack.isEmpty();
    }
}
```

Time Complexity: Amortized $O(1)$

Space Complexity: $O(n)$

29. LeetCode #225

Implement Stack using Queues

Problem:

Implement a stack using queues.

Solution Logic:

Push element and rotate queue to move last element to front.

Java Code:

```
class MyStack {
    Queue<Integer> queue = new LinkedList<>();

    public void push(int x) {
        queue.add(x);
        for (int i = 0; i < queue.size() - 1; i++) {
            queue.add(queue.remove());
        }
    }

    public int pop() {
```

```

        return queue.remove();
    }

    public int top() {
        return queue.peek();
    }

    public boolean empty() {
        return queue.isEmpty();
    }
}

```

Time Complexity:

Push: $O(n)$, Pop: $O(1)$, Top: $O(1)$

Space Complexity: $O(n)$

30. LeetCode #739

Daily Temperatures

Problem:

Given a list of daily temperatures, return a list where each element tells you how many days to wait for a warmer temperature.

Solution Logic:

Use a **monotonic stack** to store indices. Pop elements when current temp is higher, calculate difference in days.

Java Code:

```

public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] result = new int[n];
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
            int prevIndex = stack.pop();
            result[prevIndex] = i - prevIndex;
        }
        stack.push(i);
    }
    return result;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

31. LeetCode #496

Next Greater Element I

Problem:

Given two arrays, find the next greater element of each number in nums1 from nums2.

Solution Logic:

Use a map + monotonic stack to record next greater elements of nums2, then lookup for nums1.

Java Code:

```
public int[] nextGreaterElement(int[] nums1, int[] nums2) {
    Map<Integer, Integer> map = new HashMap<>();
    Stack<Integer> stack = new Stack<>();

    for (int num : nums2) {
        while (!stack.isEmpty() && num > stack.peek()) {
            map.put(stack.pop(), num);
        }
        stack.push(num);
    }

    int[] res = new int[nums1.length];
    for (int i = 0; i < nums1.length; i++) {
        res[i] = map.getOrDefault(nums1[i], -1);
    }
    return res;
}
```

Time Complexity: $O(n + m)$

Space Complexity: $O(n)$

32. LeetCode #503

Next Greater Element II

Problem:

Given a circular array, find the next greater element for every element.

Solution Logic:

Traverse array twice to simulate circular behavior. Use stack to store indices.

Java Code:

```
public int[] nextGreaterElements(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Arrays.fill(res, -1);
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < 2 * n; i++) {
        int num = nums[i % n];
        while (!stack.isEmpty() && nums[stack.peek()] < num) {
            res[stack.pop()] = num;
        }
    }
}
```

```

        if (i < n) stack.push(i);
    }
    return res;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

33. LeetCode #682

Baseball Game

Problem:

You're given a list of operations that represent scores. Implement rules for "+", "D", and "C". Return total score.

Solution Logic:

Use stack to store scores. Apply operations to modify the stack accordingly.

Java Code:

```

public int calPoints(String[] ops) {
    Stack<Integer> stack = new Stack<>();
    for (String op : ops) {
        if (op.equals("C")) {
            stack.pop();
        } else if (op.equals("D")) {
            stack.push(2 * stack.peek());
        } else if (op.equals("+")) {
            int top = stack.pop();
            int newTop = top + stack.peek();
            stack.push(top);
            stack.push(newTop);
        } else {
            stack.push(Integer.parseInt(op));
        }
    }

    int sum = 0;
    for (int score : stack) {
        sum += score;
    }
    return sum;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

34. LeetCode #150

Evaluate Reverse Polish Notation

Problem:

Evaluate the value of an arithmetic expression in Reverse Polish Notation.

Solution Logic:

Use a stack to evaluate operators as they appear. Push numbers, apply operations on the last two.

Java Code:

```
public int evalRPN(String[] tokens) {
    Stack<Integer> stack = new Stack<>();

    for (String token : tokens) {
        if ("+-*/".contains(token)) {
            int b = stack.pop();
            int a = stack.pop();
            switch (token) {
                case "+": stack.push(a + b); break;
                case "-": stack.push(a - b); break;
                case "*": stack.push(a * b); break;
                case "/": stack.push(a / b); break;
            }
        } else {
            stack.push(Integer.parseInt(token));
        }
    }
    return stack.pop();
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

35. LeetCode #239

Sliding Window Maximum

Problem:

Given an array and window size k , return max in each sliding window.

Solution Logic:

Use **Deque** to store indices in decreasing order of values. Ensure window validity.

Java Code:

```
public int[] maxSlidingWindow(int[] nums
, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.length;
```

```

int[] res = new int[n - k + 1];
Deque deque = new ArrayDeque<>();

for (int i = 0; i < n; i++) {
    while (!deque.isEmpty() && deque.peek() < i - k + 1)
        deque.poll();
    while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i])
        deque.pollLast();
    deque.offer(i);
    if (i >= k - 1)
        res[i - k + 1] = nums[deque.peek()];
}
return res;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(k)$

Trees & Binary Search Trees (BSTs)

36. LeetCode #94

Inorder Traversal (Recursive & Iterative)

Problem:

Given the root of a binary tree, return its inorder traversal.

Solution Logic:

- **Recursive:** Traverse left $\rightarrow$ root $\rightarrow$ right
- **Iterative:** Use stack to simulate recursion

Java Code (Recursive):

```

public List<Integer> inorderTraversal(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    inorder(root, res);
    return res;
}

private void inorder(TreeNode node, List<Integer> res) {
    if (node == null) return;
    inorder(node.left, res);
    res.add(node.val);
    inorder(node.right, res);
}

```

Java Code (Iterative):

```

public List<Integer> inorderTraversal(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    Stack<TreeNode> stack = new Stack<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {
        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        res.add(curr.val);
        curr = curr.right;
    }
    return res;
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

37. LeetCode #144

Preorder Traversal

Problem:

Return the preorder traversal (root $\rightarrow$ left $\rightarrow$ right) of a binary tree.

Java Code (Recursive):

```

public List<Integer> preorderTraversal(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    preorder(root, res);
    return res;
}

private void preorder(TreeNode node, List<Integer> res) {
    if (node == null) return;
    res.add(node.val);
    preorder(node.left, res);
    preorder(node.right, res);
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

38. LeetCode #145

Postorder Traversal

Problem:

Return the postorder traversal (left $\rightarrow$ right $\rightarrow$ root) of a binary tree.

Java Code (Recursive):

```
public List<Integer> postorderTraversal(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    postorder(root, res);
    return res;
}

private void postorder(TreeNode node, List<Integer> res) {
    if (node == null) return;
    postorder(node.left, res);
    postorder(node.right, res);
    res.add(node.val);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

39. LeetCode #102

Level Order Traversal (BFS)

Problem:

Return level-by-level traversal of a binary tree.

Solution Logic:

Use a queue for BFS and process each level separately.

Java Code:

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        res.add(level);
    }
    return res;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

40. LeetCode #226

Invert Binary Tree

Problem:

Invert a binary tree (mirror it).

Solution Logic:

Swap left and right subtrees recursively.

Java Code:

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;

    TreeNode left = invertTree(root.left);
    TreeNode right = invertTree(root.right);

    root.left = right;
    root.right = left;

    return root;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

41. LeetCode #104

Maximum Depth of Binary Tree

Problem:

Return the max depth (height) of a binary tree.

Solution Logic:

Recursively compute depth of left and right subtrees.

Java Code:

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$ where h is height of the tree

42. LeetCode #110

Balanced Binary Tree

Problem:

Determine if a binary tree is height-balanced.

Solution Logic:

Check left and right subtree depths recursively. If $\text{diff} > 1$, return false.

Java Code:

```
public boolean isBalanced(TreeNode root) {
    return dfs(root) != -1;
}

private int dfs(TreeNode node) {
    if (node == null) return 0;

    int left = dfs(node.left);
    int right = dfs(node.right);

    if (left == -1 || right == -1 || Math.abs(left - right) > 1)
        return -1;

    return 1 + Math.max(left, right);
}
```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

43. LeetCode #101

Symmetric Tree

Problem:

Check if a tree is a mirror of itself (symmetric).

Solution Logic:

Compare left subtree with the mirror of the right subtree.

Java Code:

```
public boolean isSymmetric(TreeNode root) {
    return isMirror(root, root);
}

private boolean isMirror(TreeNode t1, TreeNode t2) {
    if (t1 == null && t2 == null) return true;
    if (t1 == null || t2 == null) return false;
```

```

        return (t1.val == t2.val)
            && isMirror(t1.left, t2.right)
            && isMirror(t1.right, t2.left);
    }

```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

44. LeetCode #112

Path Sum

Problem:

Given a target sum, determine if there is a path from root to leaf with that sum.

Solution Logic:

At each node, subtract the current value from target. If leaf and sum equals $\rightarrow$ return true.

Java Code:

```

public boolean hasPathSum(TreeNode root, int sum) {
    if (root == null) return false;

    if (root.left == null && root.right == null)
        return sum == root.val;

    return hasPathSum(root.left, sum - root.val) ||
        hasPathSum(root.right, sum - root.val);
}

```

Time Complexity: $O(n)$

Space Complexity: $O(h)$

45. LeetCode #230

Kth Smallest Element in BST

Problem:

Find the kth smallest value in a BST.

Solution Logic:

Inorder traversal of BST gives sorted values. Return the kth.

Java Code:

```

public int kthSmallest(TreeNode root, int k) {
    Stack<TreeNode> stack = new Stack<>();
    TreeNode curr = root;

    while (curr != null || !stack.isEmpty()) {

```

```

        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        k--;
        if (k == 0) return curr.val;
        curr = curr.right;
    }
    return -1;
}

```

Time Complexity: $O(h + k)$

Space Complexity: $O(h)$

Great! Let's continue with:

Binary Search & Sorting

46. LeetCode #704

Binary Search

Problem:

Given a sorted array and a target value, return the index if found, else -1.

Solution Logic:

Standard binary search using two pointers `left` and `right`.

Java Code:

```

public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

47. LeetCode #35

Search Insert Position

Problem:

Find the index where target should be inserted to maintain sorted order.

Java Code:

```
public int searchInsert(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return left;
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

48. LeetCode #153

Find Minimum in Rotated Sorted Array

Problem:

Find the minimum element in a rotated sorted array (no duplicates).

Solution Logic:

Use binary search; check mid vs right.

Java Code:

```
public int findMin(int[] nums) {
    int left = 0, right = nums.length - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] > nums[right]) left = mid + 1;
        else right = mid;
    }
    return nums[left];
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

49. LeetCode #33

Search in Rotated Sorted Array

Problem:

Given a rotated sorted array, search for a target value.

Java Code:

```
public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;

        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        } else {
            if (nums[mid] < target && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

50. LeetCode #34

Find First and Last Position of Element in Sorted Array

Problem:

Return the first and last index of the target value.

Solution Logic:

Use binary search twice: once for left bound, once for right.

Java Code:

```
public int[] searchRange(int[] nums, int target) {
    int[] res = {-1, -1};
    res[0] = findBound(nums, target, true);
    res[1] = findBound(nums, target, false);
    return res;
}

private int findBound(int[] nums, int target, boolean isFirst) {
    int left = 0, right = nums.length - 1, result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            if (isFirst) result = mid;
            else result = mid;
        }
        if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return result;
}
```

```

        result = mid;
        if (isFirst) right = mid - 1;
        else left = mid + 1;
    } else if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
}
return result;
}

```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

51. LeetCode #88

Merge Sorted Arrays

Problem:

Merge `nums2` into `nums1` (in-place) assuming enough space.

Java Code:

```

public void merge(int[] nums1, int m, int[] nums2, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;

    while (i >= 0 && j >= 0) {
        if (nums1[i] > nums2[j])
            nums1[k--] = nums1[i--];
        else
            nums1[k--] = nums2[j--];
    }

    while (j >= 0)
        nums1[k--] = nums2[j--];
}

```

Time Complexity: $O(m + n)$

Space Complexity: $O(1)$

52. LeetCode #56

Merge Intervals

Problem:

Merge all overlapping intervals.

Java Code:

```

public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> res = new ArrayList<>();
}

```

```

        for (int[] interval : intervals) {
            if (res.isEmpty() || res.get(res.size() - 1)[1] < interval[0])
                res.add(interval);
            else
                res.get(res.size() - 1)[1] = Math.max(res.get(res.size() - 1)[1],
interval[1]);
        }
        return res.toArray(new int[res.size()][]);
    }
}

```

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

53. LeetCode #215

Kth Largest Element in Array

Problem:

Find the kth largest element using a heap.

Java Code:

```

public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k)
            minHeap.poll();
    }
    return minHeap.peek();
}

```

Time Complexity: $O(n \log k)$

Space Complexity: $O(k)$

54. LeetCode #912

Sort an Array (Merge Sort)

Problem:

Sort the array using merge sort.

Java Code:

```

public int[] sortArray(int[] nums) {
    if (nums.length <= 1) return nums;

    int mid = nums.length / 2;
    int[] left = sortArray(Arrays.copyOfRange(nums, 0, mid));
    int[] right = sortArray(Arrays.copyOfRange(nums, mid, nums.length));
}

```

```

        return merge(left, right);
    }

    private int[] merge(int[] left, int[] right) {
        int[] res = new int[left.length + right.length];
        int i = 0, j = 0, k = 0;

        while (i < left.length && j < right.length)
            res[k++] = left[i] < right[j] ? left[i++] : right[j++];

        while (i < left.length) res[k++] = left[i++];
        while (j < right.length) res[k++] = right[j++];

        return res;
    }

```

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

55. LeetCode #347

Top K Frequent Elements

Problem:

Return the k most frequent elements in the array.

Solution Logic:

Use a frequency map + min-heap.

Java Code:

```

public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums)
        freq.put(num, freq.getOrDefault(num, 0) + 1);

    PriorityQueue<Map.Entry<Integer, Integer>> heap =
        new PriorityQueue<>((a, b) -> a.getValue() - b.getValue());

    for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
        heap.offer(entry);
        if (heap.size() > k)
            heap.poll();
    }

    int[] res = new int[k];
    for (int i = 0; i < k; i++)
        res[i] = heap.poll().getKey();

    return res;
}

```

Time Complexity: $O(n \log k)$

Space Complexity: $O(n)$

Great! Let's continue with:

□ Topic 7: Dynamic Programming

Questions 56–75

(Java | Optimized | Placement-Focused | Includes Explanation, Code, Time & Space)

56. LeetCode #509

Fibonacci Number (Memoization / DP)

Problem:

Compute the n th Fibonacci number.

$\text{fib}(0) = 0, \text{fib}(1) = 1$

Solution Logic:

Use bottom-up DP with tabulation to avoid recursion stack.

Java Code:

```
public int fib(int n) {
    if (n < 2) return n;
    int[] dp = new int[n + 1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

57. LeetCode #70

Climbing Stairs

Problem:

You can climb either 1 or 2 steps. How many distinct ways to reach the top?

Solution Logic:

Fibonacci-like: $\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$

Java Code:

```
public int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int temp = a + b;
        a = b;
        b = temp;
    }
    return b;
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

58. LeetCode #64

Minimum Path Sum

Problem:

Find the minimum sum path from top-left to bottom-right in a grid.

Solution Logic:

$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$

Java Code:

```
public int minPathSum(int[][] grid) {
    int m = grid.length, n = grid[0].length;

    for (int i = 1; i < m; i++)
        grid[i][0] += grid[i - 1][0];
    for (int j = 1; j < n; j++)
        grid[0][j] += grid[0][j - 1];

    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            grid[i][j] += Math.min(grid[i - 1][j], grid[i][j - 1]);

    return grid[m - 1][n - 1];
}
```

Time Complexity: $O(m \times n)$

Space Complexity: $O(1)$ (modifies input)

59. LeetCode #300

Longest Increasing Subsequence (LIS)

Problem:

Find the length of the longest strictly increasing subsequence in an array.

Java Code:

```
public int lengthOfLIS(int[] nums) {
    ArrayList<Integer> sub = new ArrayList<>();
    for (int num : nums) {
        int i = Collections.binarySearch(sub, num);
        if (i < 0) i = -(i + 1);
        if (i < sub.size()) sub.set(i, num);
        else sub.add(num);
    }
    return sub.size();
}
```

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

60. LeetCode #322

Coin Change (Unbounded Knapsack)

Problem:

Given `coins[]` and `amount`, return the fewest coins to make the amount.

Java Code:

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;

    for (int coin : coins) {
        for (int i = coin; i <= amount; i++) {
            dp[i] = Math.min(dp[i], dp[i - coin] + 1);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Time Complexity: $O(\text{amount} \times n)$

Space Complexity: $O(\text{amount})$

61. LeetCode #516

Longest Palindromic Subsequence

Problem:

Return the length of the longest palindromic subsequence in a string.

Java Code:

```
public int longestPalindromeSubseq(String s) {
    int n = s.length();
    int[][] dp = new int[n][n];

    for (int i = n - 1; i >= 0; i--) {
        dp[i][i] = 1;
        for (int j = i + 1; j < n; j++) {
            if (s.charAt(i) == s.charAt(j))
                dp[i][j] = dp[i + 1][j - 1] + 2;
            else
                dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
        }
    }
    return dp[0][n - 1];
}
```

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

62. LeetCode #72**Edit Distance (Levenshtein Distance)****Problem:**

Return min operations to convert word1 to word2 (insert, delete, replace).

Java Code:

```
public int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 0; i <= m; i++)
        dp[i][0] = i;
    for (int j = 0; j <= n; j++)
        dp[0][j] = j;

    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (word1.charAt(i - 1) == word2.charAt(j - 1))
                dp[i][j] = dp[i - 1][j - 1];
            else
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],
                                         Math.min(dp[i - 1][j], dp[i][j - 1]));

    return dp[m][n];
}
```

Time Complexity: $O(m \times n)$

Space Complexity: $O(m \times n)$

63. LeetCode #91

Decode Ways

Problem:

Return the number of ways to decode a digit string ('1' -> A, '26' -> Z).

Java Code:

```
public int numDecodings(String s) {
    if (s.charAt(0) == '0') return 0;

    int n = s.length();
    int[] dp = new int[n + 1];
    dp[0] = 1; dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        int one = Integer.parseInt(s.substring(i - 1, i));
        int two = Integer.parseInt(s.substring(i - 2, i));

        if (one >= 1 && one <= 9)
            dp[i] += dp[i - 1];
        if (two >= 10 && two <= 26)
            dp[i] += dp[i - 2];
    }
    return dp[n];
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

64. LeetCode #329

Longest Increasing Path in a Matrix

Problem:

Find the longest increasing path in a matrix. Move up/down/left/right.

Java Code:

```
int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};
int m, n;

public int longestIncreasingPath(int[][] matrix) {
    if (matrix.length == 0) return 0;
    m = matrix.length; n = matrix[0].length;
```

```

int[][] memo = new int[m][n];
int max = 0;

for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
        max = Math.max(max, dfs(matrix, i, j, memo));
return max;
}

private int dfs(int[][] mat, int i, int j, int[][] memo) {
    if (memo[i][j] != 0) return memo[i][j];
    int max = 1;
    for (int[] dir : dirs) {
        int x = i + dir[0], y = j + dir[1];
        if (x >= 0 && y >= 0 && x < m && y < n && mat[x][y] > mat[i][j])
            max = Math.max(max, 1 + dfs(mat, x, y, memo));
    }
    memo[i][j] = max;
    return max;
}

```

Time Complexity: $O(m \times n)$

Space Complexity: $O(m \times n)$

65. LeetCode #221

Maximal Square

Problem:

Find the largest square of 1's in a binary matrix.

```

```java
public int maximalSquare(char[][] matrix) {
 int m = matrix.length, n = matrix[0].length;
 int[][] dp = new int[m + 1][n + 1];
 int max = 0;

 for (int i = 1; i <= m; i++)
 for (int j = 1; j <= n; j++)
 if (matrix[i - 1][j - 1] == '1') {
 dp[i][j] = Math.min(
 Math.min(dp[i - 1][j], dp[i][j - 1]),
 dp[i - 1][j - 1]) + 1;
 max = Math.max(max, dp[i][j]);
 }
 return max * max;
}

```

**Time Complexity:**  $O(m \times n)$

**Space Complexity:**  $O(m \times n)$

# Advanced Dynamic Programming / Graphs

---

## 66. LeetCode #131

### Palindrome Partitioning

#### Problem:

Partition a string such that every substring is a palindrome. Return all such possible partitionings.

#### Solution Logic:

Use backtracking with a DP table to precompute palindromic substrings.

#### Java Code:

```
public List<List<String>> partition(String s) {
 List<List<String>> result = new ArrayList<>();
 int n = s.length();
 boolean[][] dp = new boolean[n][n];

 for (int i = n - 1; i >= 0; i--)
 for (int j = i; j < n; j++)
 dp[i][j] = (s.charAt(i) == s.charAt(j)) && (j - i < 2 || dp[i + 1][j - 1]);

 backtrack(s, 0, new ArrayList<>(), result, dp);
 return result;
}

private void backtrack(String s, int start, List<String> path, List<List<String>> res, boolean[][] dp) {
 if (start == s.length()) {
 res.add(new ArrayList<>(path));
 return;
 }
 for (int end = start; end < s.length(); end++) {
 if (dp[start][end]) {
 path.add(s.substring(start, end + 1));
 backtrack(s, end + 1, path, res, dp);
 path.remove(path.size() - 1);
 }
 }
}
```

**Time Complexity:** Exponential

**Space Complexity:**  $O(n^2)$  (for DP table + recursion)

---

## 67. LeetCode #76

### Minimum Window Substring

#### Problem:

Given strings  $s$  and  $t$ , return the minimum window in  $s$  which contains all characters of  $t$ .

#### Solution Logic:

Use sliding window and frequency maps to track required and current characters.

#### Java Code:

```
public String minWindow(String s, String t) {
 if (s.length() < t.length()) return "";

 Map<Character, Integer> map = new HashMap<>();
 for (char c : t.toCharArray())
 map.put(c, map.getOrDefault(c, 0) + 1);

 int start = 0, end = 0, minLen = Integer.MAX_VALUE, match = 0, minStart = 0;
 Map<Character, Integer> window = new HashMap<>();

 while (end < s.length()) {
 char c = s.charAt(end++);
 window.put(c, window.getOrDefault(c, 0) + 1);

 if (map.containsKey(c) && window.get(c).intValue() ==
 map.get(c).intValue())
 match++;

 while (match == map.size()) {
 if (end - start < minLen) {
 minLen = end - start;
 minStart = start;
 }
 char left = s.charAt(start++);
 window.put(left, window.get(left) - 1);
 if (map.containsKey(left) && window.get(left) < map.get(left))
 match--;
 }
 }
 return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart, minStart +
 minLen);
}
```

**Time Complexity:**  $O(|S| + |T|)$

**Space Complexity:**  $O(|S| + |T|)$

---

## 68. LeetCode #212

### Word Search II

#### Problem:

Given a board and a list of words, find all words present in the board. You may move in 4 directions and can't revisit cells.

#### Solution Logic:

Use Trie + DFS from each cell. Mark visited to prevent revisiting.

#### Java Code:

```
class TrieNode {
 Map<Character, TrieNode> children = new HashMap<>();
 String word = null;
}

private TrieNode buildTrie(String[] words) {
 TrieNode root = new TrieNode();
 for (String word : words) {
 TrieNode node = root;
 for (char c : word.toCharArray())
 node = node.children.computeIfAbsent(c, k -> new TrieNode());
 node.word = word;
 }
 return root;
}

public List<String> findWords(char[][] board, String[] words) {
 List<String> res = new ArrayList<>();
 TrieNode root = buildTrie(words);
 int m = board.length, n = board[0].length;

 for (int i = 0; i < m; i++)
 for (int j = 0; j < n; j++)
 dfs(board, i, j, root, res);

 return res;
}

private void dfs(char[][] board, int i, int j, TrieNode node, List<String> res) {
 char c = board[i][j];
 if (c == '#' || !node.children.containsKey(c)) return;

 node = node.children.get(c);
 if (node.word != null) {
 res.add(node.word);
 node.word = null;
 }

 board[i][j] = '#';
 for (int[] d : new int[][]{{0,1},{1,0},{0,-1},{-1,0}})
 if (i + d[0] >= 0 && i + d[0] < board.length && j + d[1] >= 0 && j + d[1] <
board[0].length)
 dfs(board, i + d[0], j + d[1], node, res);
 board[i][j] = c;
}
```

```
}
```

**Time Complexity:**  $O(M \times N \times 4^L)$

**Space Complexity:**  $O(K \times L)$  where  $K = \text{\#words}$ ,  $L = \text{avg word length}$

---

## 69. LeetCode #312

### Burst Balloons

#### Problem:

Given an array `nums`, burst balloons to maximize coins. You get `nums[i - 1] * nums[i] * nums[i + 1]` for bursting `i`.

#### Solution Logic:

DP on intervals: pick the last balloon to burst between `i` and `j`.

#### Java Code:

```
public int maxCoins(int[] nums) {
 int n = nums.length;
 int[] newNums = new int[n + 2];
 newNums[0] = newNums[n + 1] = 1;
 for (int i = 0; i < n; i++) newNums[i + 1] = nums[i];

 int[][] dp = new int[n + 2][n + 2];
 for (int len = 2; len <= n + 1; len++) {
 for (int left = 0; left <= n + 1 - len; left++) {
 int right = left + len;
 for (int k = left + 1; k < right; k++) {
 dp[left][right] = Math.max(dp[left][right],
 newNums[left] * newNums[k] * newNums[right] + dp[left][k] +
 dp[k][right]);
 }
 }
 }
 return dp[0][n + 1];
}
```

**Time Complexity:**  $O(n^3)$

**Space Complexity:**  $O(n^2)$

---

## 70. LeetCode #10

### Regular Expression Matching

#### Problem:

Implement regex matching with `.` (any char) and `*` (zero or more of previous char).

**Solution Logic:**

DP:  $dp[i][j]$  = match between  $s[0...i-1]$  and  $p[0...j-1]$

**Java Code:**

```
public boolean isMatch(String s, String p) {
 int m = s.length(), n = p.length();
 boolean[][] dp = new boolean[m + 1][n + 1];
 dp[0][0] = true;

 for (int j = 2; j <= n; j++)
 if (p.charAt(j - 1) == '*')
 dp[0][j] = dp[0][j - 2];

 for (int i = 1; i <= m; i++) {
 for (int j = 1; j <= n; j++) {
 if (p.charAt(j - 1) == '.' || p.charAt(j - 1) == s.charAt(i - 1))
 dp[i][j] = dp[i - 1][j - 1];
 else if (p.charAt(j - 1) == '*') {
 dp[i][j] = dp[i][j - 2];
 if (p.charAt(j - 2) == '.' || p.charAt(j - 2) == s.charAt(i - 1))
 dp[i][j] |= dp[i - 1][j];
 }
 }
 }
 return dp[m][n];
}
```

**Time Complexity:**  $O(m \times n)$

**Space Complexity:**  $O(m \times n)$

## **Backtracking & Advanced Recursion**

### **71. LeetCode #37**

#### **Sudoku Solver**

**Problem Statement:**

Write a program to solve a Sudoku puzzle by filling the empty cells. A valid solution satisfies the rules of Sudoku: each row, column, and 3×3 box must contain digits 1–9 without repetition.

**Solution Logic:**

Use backtracking to try placing digits 1–9 in empty cells. Check row, column, and box constraints before placing.

**Java Code:**

```
public void solveSudoku(char[][] board) {
 solve(board);
}
```

```

 }

 private boolean solve(char[][] board) {
 for (int i = 0; i < 9; i++) {
 for (int j = 0; j < 9; j++) {
 if (board[i][j] == '.') {
 for (char c = '1'; c <= '9'; c++) {
 if (isValid(board, i, j, c)) {
 board[i][j] = c;
 if (solve(board)) return true;
 board[i][j] = '.';
 }
 }
 return false;
 }
 }
 }
 return true;
 }

 private boolean isValid(char[][] board, int row, int col, char c) {
 for (int i = 0; i < 9; i++) {
 if (board[i][col] == c || board[row][i] == c) return false;
 int boxRow = 3 * (row / 3) + i / 3;
 int boxCol = 3 * (col / 3) + i % 3;
 if (board[boxRow][boxCol] == c) return false;
 }
 return true;
 }
}

```

**Time Complexity:**  $O(9^n)$  (backtracking)

**Space Complexity:**  $O(n^2)$  for the board (in-place)

---

## 72. LeetCode #51

### N-Queens

#### Problem Statement:

Place N queens on an  $N \times N$  chessboard so that no two queens attack each other. Return all distinct solutions.

#### Solution Logic:

Backtracking: Try placing queens row by row, checking column and diagonal conflicts.

#### Java Code:

```

public List<List<String>> solveNQueens(int n) {
 List<List<String>> res = new ArrayList<>();
 char[][] board = new char[n][n];
 for (char[] row : board) Arrays.fill(row, '.');
 backtrack(res, board, 0);
 return res;
}

```

```

private void backtrack(List<List<String>> res, char[][] board, int row) {
 if (row == board.length) {
 List<String> solution = new ArrayList<>();
 for (char[] line : board)
 solution.add(new String(line));
 res.add(solution);
 return;
 }

 for (int col = 0; col < board.length; col++) {
 if (isSafe(board, row, col)) {
 board[row][col] = 'Q';
 backtrack(res, board, row + 1);
 board[row][col] = '.';
 }
 }
}

private boolean isSafe(char[][] board, int row, int col) {
 for (int i = 0; i < row; i++)
 if (board[i][col] == 'Q') return false;

 for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
 if (board[i][j] == 'Q') return false;

 for (int i = row - 1, j = col + 1; i >= 0 && j < board.length; i--, j++)
 if (board[i][j] == 'Q') return false;

 return true;
}

```

**Time Complexity:**  $O(N!)$

**Space Complexity:**  $O(N^2)$

---

## 73. LeetCode #52

### N-Queens II

#### Problem Statement:

Return the number of distinct solutions to the N-Queens puzzle (only count, no board required).

#### Solution Logic:

Same as N-Queens but optimized for count using bitsets or sets.

#### Java Code:

```

public int totalNQueens(int n) {
 return countQueens(0, n, new HashSet<>(), new HashSet<>(), new HashSet<>());
}

private int countQueens(int row, int n, Set<Integer> cols, Set<Integer> d1,
 Set<Integer> d2) {
 if (row == n) return 1;

```

```

 int count = 0;
 for (int col = 0; col < n; col++) {
 int diag1 = row - col, diag2 = row + col;
 if (cols.contains(col) || d1.contains(diag1) || d2.contains(diag2))
 continue;
 cols.add(col); d1.add(diag1); d2.add(diag2);
 count += countQueens(row + 1, n, cols, d1, d2);
 cols.remove(col); d1.remove(diag1); d2.remove(diag2);
 }
 return count;
}

```

**Time Complexity:**  $O(N!)$

**Space Complexity:**  $O(N)$

---

## 74. LeetCode #77

### Combinations

#### Problem Statement:

Return all possible combinations of  $k$  numbers out of the range  $[1, n]$ .

#### Solution Logic:

Use backtracking to build combinations by choosing or skipping each number.

#### Java Code:

```

public List<List<Integer>> combine(int n, int k) {
 List<List<Integer>> res = new ArrayList<>();
 backtrack(res, new ArrayList<>(), 1, n, k);
 return res;
}

private void backtrack(List<List<Integer>> res, List<Integer> path, int start, int
n, int k) {
 if (path.size() == k) {
 res.add(new ArrayList<>(path));
 return;
 }
 for (int i = start; i <= n; i++) {
 path.add(i);
 backtrack(res, path, i + 1, n, k);
 path.remove(path.size() - 1);
 }
}

```

**Time Complexity:**  $O(C(n, k))$

**Space Complexity:**  $O(k)$  recursion stack

---

## 75. LeetCode #39

## Combination Sum

### Problem Statement:

Given an array of distinct integers and a target, return all combinations where elements sum to the target. You may reuse elements.

### Solution Logic:

Use backtracking and explore including current number multiple times.

### Java Code:

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
 List<List<Integer>> res = new ArrayList<>();
 backtrack(res, new ArrayList<>(), candidates, target, 0);
 return res;
}

private void backtrack(List<List<Integer>> res, List<Integer> temp, int[] nums, int
remain, int start) {
 if (remain == 0) {
 res.add(new ArrayList<>(temp));
 return;
 }
 if (remain < 0) return;

 for (int i = start; i < nums.length; i++) {
 temp.add(nums[i]);
 backtrack(res, temp, nums, remain - nums[i], i); // not i + 1 because we
can reuse
 temp.remove(temp.size() - 1);
 }
}
```

**Time Complexity:**  $O(2^t)$ , where  $t$  = target

**Space Complexity:**  $O(\text{target})$

---

Congratulations! You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills. Stay connected with us for more coding tutorials, challenges, tips, and tech content on Instagram: @SYNTAXERROR. Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved. This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.